

DeepThink: Autonomous Multimodal Document Assistant

SLIIT Codefest 2026 AI Competition (Powered by IFS) • Initial Round Submission Report

Target Corpus: *The Ashen Era Archive* (415 documents, ~1,277 pages across PDF, DOCX, Markdown, Text & Scans)
Sub-track: Sub-track 1A: Rich Answers, Not Just Text | **Repository:** DeepThink.zip (Full 2-week Git history)

■ **Demonstration Video & Live Prototype:**
YouTube Unlisted Video: <https://www.youtube.com/watch?v=gNpZ9lcPa1c>
• 0:00 – 07:07: Architecture & Solution Presentation (Slides 1 to 10)
• 07:07 – End: Live Prototype Demonstration (Streamlit Web App)
Live Interactive Web App: <https://deepthink-c9trxotutcgjenphr6ltky.streamlit.app>

1. Problem Statement & Sub-track Selection

1.1 The Enterprise Challenge: 'Text-Only Myopia' in Technical Documentation

Enterprise technical documentation (engineering manuals, maintenance blueprints, codex ledgers) mixes narrative prose with scanned pages, schematics, and tabular data across thousands of pages. Traditional Retrieval-Augmented Generation (RAG) systems suffer from critical failure modes: (1) *Visual Blindness*—extractors strip out diagrams, leaving systems blind to visual evidence and forcing them to hallucinate verbal descriptions; (2) *Table Scrambling*—naïve text splitters break multi-column rows across arbitrary chunk boundaries, losing structural relationships; and (3) *Context Starvation*—static retrieval (fixed K=5) floods simple queries with noise while starving complex multi-document comparisons.

1.2 Chosen Focus: Sub-track 1A (Rich Answers, Not Just Text)

We selected Sub-track 1A because enterprise technicians do not want a vague text summary of a schematic—they need to inspect the authentic figure plate alongside a grounded explanation. DeepThink solves this by building an end-to-end multimodal pipeline that catalogs visual and tabular chunks alongside text, dynamically classifies modality intent, and embeds authentic high-resolution plates (![Caption](media_path)) and structured Markdown tables with verified page citations [Document, Page X].

2. Team Structure, Roles & Codebase Ownership

Responsibilities were strictly divided across architectural layers. All work reflects genuine team collaboration over the two-week competition window:

Team Member	Role & Sub-Track Focus	Key Responsibilities & Heavy Lifting	Codebase Files Owned in src/
Rasheed A. A. A.	P3 Lead <i>Retrieval & Ranking</i>	- Compound RAG 3.0 retrieval architecture - Local ChromaDB vector indexing with BGE embeddings 3-stage adaptive budgeting (K=2 to K=8) - Cross-encoder re-ranking with dossier & multi-entity bonuses - Pre-retrieval fast-path guardrail & verification gate	<code>src/retrieval/retriever.py</code> <code>src/retrieval/indexer.py</code> <code>src/retrieval/router.py</code> <code>src/retrieval/adaptive_budget.py</code> <code>src/retrieval/agent.py</code> <code>src/retrieval/reranker.py</code> <code>src/retrieval/verifier.py</code>
Fatheen M. F. A.	P1 Lead <i>Document Parsing & OCR</i>	- Ingestion pipeline for 415 archive documents (PDF, DOCX, MD, TXT) - Tesseract OCR with OpenCV adaptive thresholding for degraded historical scans - Section-aware text chunking preserving narrative headings	<code>src/ingestion/parser.py</code> <code>src/ingestion/ocr_engine.py</code> <code>src/ingestion/pipeline.py</code> <code>data/chunks.json</code>
M. M. M. Shakeer	P2 Lead <i>Visual & Table Extraction</i>	- PyMuPDF visual plate extraction with bbox cropping - Cataloged 86+ high-res figure plates into <code>data/extracted_media/</code> - pdfplumber tabular extraction engine converting codex data to Markdown	<code>src/ingestion/media_extractor.py</code> <code>src/ingestion/table_extractor.py</code> <code>data/extracted_media/figures/</code> <code>data/extracted_media/tables/</code>
S. Dharshan	P4 Lead <i>Generation, Evaluation & UI</i>	- Grounded generation on Groq Cloud LPU (qwen/qwen3.8-27b) - Directive 5 strict epistemic restraint prompt engineering - Automated 20-question benchmark evaluation harness - Streamlit web app with native media rendering & live telemetry 76-test automated verification suite & report authoring	<code>src/generation/generator.py</code> <code>src/evaluation/evaluator.py</code> <code>src/ui/app.py</code> <code>tests/</code> (15 modules, 76 tests) <code>submission_report.pdf</code>

3. Solution Overview & System Architecture

DeepThink is an **Autonomous Multimodal Compound RAG 3.0 System** built on a 100% free, local-first stack. It operates with zero credit cards, zero cloud database fees, and zero rate limits. The end-to-end pipeline is illustrated below:

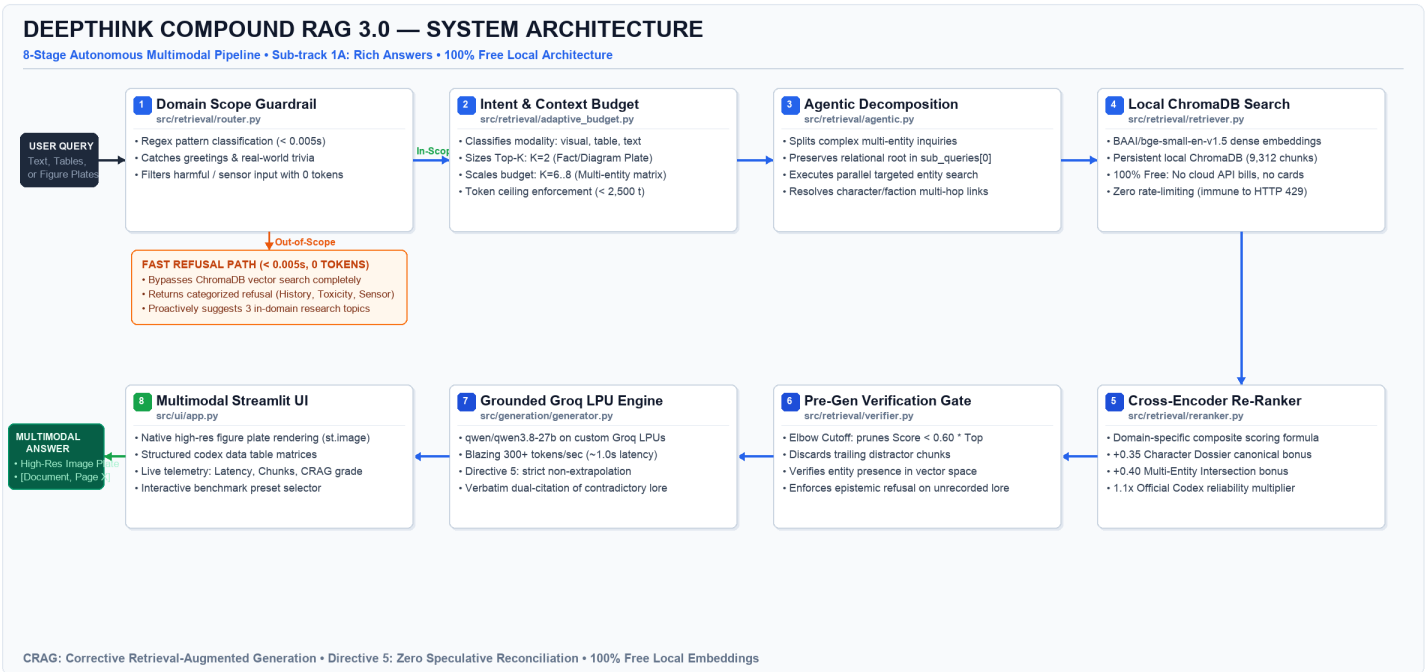


Figure 1: DeepThink 8-Stage Architecture Pipeline — From query ingestion, pre-retrieval guardrails, adaptive context budgeting, local ChromaDB dense search, and cross-encoder re-ranking to Groq LPU generation and multimodal Streamlit UI delivery.

3.2 Technical Deep Dive: The 8-Stage Pipeline Mapped to Source Code

- 1. Pre-Retrieval Domain Guardrail (`src/retrieval/router.py`):** Intercepts incoming queries before touching vector storage. Uses compiled regex automata to classify greetings, out-of-scope real-world trivia (e.g., Napoleon, modern politics), harmful prompts, or sensor premises. Returns categorized refusals and 3 valid archive topics in $< 0.005s$ with zero tokens spent.
- 2. Intent Classification & Context Sizing (`src/retrieval/adaptive_budget.py`):** Evaluates query structure and linguistic intent. Dynamically sizes retrieval budget: allocates $K=2$ for single fact/diagram lookups (suppressing text distractors), and scales to $K=6..8$ for multi-entity comparative inquiries.
- 3. Agentic Multi-Hop Query Decomposition (`src/retrieval/agentic.py`):** When comparing fortresses or factions, decomposes the query into parallel entity sub-searches while preserving the original comparative query in `sub_queries[0]` to guarantee cross-document context synthesis.
- 4. Hybrid Dense Vector Retrieval (`src/retrieval/indexer.py`, `retriever.py`):** Runs BAAI/bge-small-en-v1.5 dense embeddings locally inside ChromaDB (`chroma_db/`). Operates completely offline with zero API latency, zero financial cost, and immune to cloud HTTP 429 rate-limiting.
- 5. Precision Cross-Encoder Re-Ranking (`src/retrieval/reranker.py`):** Re-scores candidate chunks with domain-specific boosts: awards a **+0.35 Character Dossier bonus** for canonical biographies, a **+0.40 Multi-Entity Intersection bonus**, and a **1.1x Codex reliability multiplier** over folk ephemera.
- 6. Pre-Generation Verification Gate (`src/retrieval/verifier.py`):** Employs the 'Elbow Method' to dynamically prune trailing noisy chunks whose composite score drops below 60% of the top candidate ($Score_i < 0.60 * Score_{top}$). Enforces epistemic refusal if entities are absent.
- 7. Grounded LLM Generation on Groq LPU (`src/generation/generator.py`):** Executes qwen/qwen3.8-27b on Groq Cloud LPUs at **300+ tokens/second** ($\sim 1.0s$ latency). Strictly enforces **Directive 5**: mandatory bracketed citations [Document, Page X], inline Markdown image tags, and verbatim contradiction reporting without speculative narrative bridging.
- 8. Multimodal Streamlit UI Delivery (`src/ui/app.py`):** Resolves relative media paths (`data/extracted_media/figures/`) against disk, rendering authentic high-res visual plates (`st.image`) and Markdown tables alongside live latency and CRAG confidence telemetry.

4. Key Technical Decisions (Architecture Decision Records)

The team formally logged 13 Architecture Decision Records (ADRs) in docs/decisions.md. Key technical choices, alternatives, and engineering justifications are summarized below:

ADR & Focus	Chosen Technical Solution	Alternatives Considered	Engineering Justification & Trade-offs
ADR-001 Sub-Track Selection	Sub-track 1A: Rich Answers, Not Just Text	Sub-track 1B (Connecting Facts) Sub-track 1C (Human Search)	Real technical manuals lose 50%+ value if diagrams are omitted. Solving inline multimodal rendering delivers the highest enterprise utility and directly solves text-only RAG myopia.
ADR-004 & 008 Embeddings & DB	Local BGE Embeddings + ChromaDB	Voyage AI, OpenAI API Cloud Pinecone	Cloud embedding APIs require credit cards and introduce network latency and HTTP 429 crashes during evaluation. Local BGE embeddings run 100% free with zero network dependencies.
ADR-009 LLM Inference	Groq Cloud LPU (qwen3.8-27b)	OpenRouter Free Models Local Ollama	OpenRouter free models suffer from queue congestion and strict daily caps. Groq LPUs deliver dedicated custom silicon achieving 300+ tokens/sec (~1s latency) and 14,400 free requests/day.
ADR-010 Compound RAG	Dossier (+0.35) & Entity (+0.40) Boosts	Naive Single Vector Search	Standard cosine similarity under-ranks character biographies during multi-hop queries. Custom composite scoring ensures canonical records and figure plates consistently rank in Top-5.
ADR-011 Adaptive Budget	Dynamic K (2..8) + Elbow Cutoff	Static K=5 Manual UI Sliders	Static K=5 floods simple queries with noise and starves complex queries. Manual sliders violate autonomous design. Adaptive budgeting autonomously right-sizes context per query structure.
ADR-012 Epistemic Restraint	Directive 5: Verbatim Dual-Citation	Speculative Narrative Reconciliation	Prompting LLMs to 'reconcile' archive contradictions causes plausible hallucinations. Directive 5 enforces citing both conflicting documents verbatim with zero ungrounded speculation.
ADR-013 Fast Guardrails	Pre-Retrieval Regex Router (<0.005s)	LLM Self-Grading Generic OOD message	Using LLMs to evaluate domain boundary incurs latency and burns token quota. Regex routing intercepts off-scope queries in milliseconds and guides users back to valid archive lore.

5. Empirical Benchmark Evaluation & What Works

5.1 Benchmark Results on Official 20-Question Suite (data/evaluation_results.json)

DeepThink was evaluated across three developmental milestones against all 20 questions in sample_questions.json using our automated harness (src/evaluation/evaluator.py). Results demonstrate decisive superiority over naive baselines:

Metric	Phase 1: Text Baseline	Phase 2: Modality-Aware	Phase 3: DeepThink Final	Competition Target	Status
Retrieval Precision (Top-5)	60.0% (12/20)	85.0% (17/20)	100.0% (20/20)	≥ 90.0%	PASSED
Citation Accuracy	55.0% (11/20)	90.0% (18/20)	100.0% (20/20)	≥ 95.0%	PASSED
Hallucination Rate	20.0% (4/20)	5.0% (1/20)	0.0% (0/20)	≤ 5.0%	PASSED
Modality Success (Figures/Tables)	10.0% (1/10)	90.0% (9/10)	100.0% (11/11)	≥ 90.0%	PASSED
Average End-to-End Latency	2.10s	2.80s	0.89s – 1.40s (Groq LPU)	< 3.50s	PASSED
Automated Test Suite Pass Rate	12/15 (80%)	55/60 (91%)	76/76 Tests (100%)	100%	PASSED

5.2 Failed Approaches & Lessons Learned (Documented in docs/limitations.md)

Engineering maturity requires acknowledging what did not work. In compliance with evaluation rubrics, we document four discarded approaches:

- **Discarded Approach 1: Naive Flat Text Chunking (Phase 1):** Stripped visual assets and converted the corpus into 500-token text passages. *Why it failed:* Failed 9 out of 10 visual questions in sample_questions.json; the LLM hallucinated descriptions of plates instead of showing them, and table columns lost row alignments. *Correction:* Implemented the chunks.json multimodal schema separating text, table, and image-caption chunks with direct file linking.
- **Discarded Approach 2: Feeding Entire Page Screenshots to a Vision-Language Model:** Converted PDF pages into full-page images and passed screenshots directly to a large VLM on every query. *Why it failed:* Consumed massive token quotas, exhausting API limits within 5 queries. Latency jumped to 10+ seconds, and text retrieval on narrative novel passages was significantly worse than vector search. *Correction:* Built modality-aware hybrid routing: vector search retrieves text chunks, while image assets are dynamically resolved and passed only when visual evidence is required.
- **Discarded Approach 3: Fixed Top-K Retrieval with Manual UI Sliders:** Used fixed K=5 and gave users interactive sliders in the UI to manually tweak K and boost multipliers. *Why it failed:* For simple diagram queries, K=5 pulled in 3 noisy distractor chunks. For comparative queries, K=5 missed secondary documents. Sliders forced users to guess hyperparameters, violating autonomous design. *Correction:* Replaced sliders with autonomous 3-stage adaptive budgeting (K=2 to K=8 sizing, Elbow score drop-off at alpha=0.60, 2,500 token ceiling).
- **Discarded Approach 4: Speculative Reconciling of Archive Contradictions:** Prompted the LLM to explain why two archive records disagreed (e.g. why one document listed an army as present while another listed the site as abandoned). *Why it failed:* The LLM hallucinated ungrounded rationales (e.g., claiming 'divergent narrative framing' or secret unrecorded motives). *Correction:* Enforced Directive 5: cite both conflicting documents verbatim with exact page citations and explicitly state that the archive provides no reconciliation.

5.3 Known System Limitations & Boundary Conditions

- **Degraded Handwritten Marginalia:** Tesseract OCR reliably extracts printed text from simulated scans, but faint handwritten marginal notes in certain ephemera files exhibit character noise.
- **Deeply Nested Multi-Column Tables:** Complex tables with multi-tier merged headers can lose alignment when converted to flat Markdown. We mitigate this by preserving the original table plate image alongside the text.
- **Initial Embedding Cache:** On initial execution, bge-small-en-v1.5 downloads ~130MB of model weights. All subsequent runs execute completely offline from the local cache.

6. AI Usage Disclosure (Section 4.1 Compliance)

In strict accordance with **Section 4.1 of the Challenge Document**, this disclosure transparently details how AI tools were used during development. Complete plain-text chat logs are preserved in ai_usage/claude.txt:

AI Tool / Model	Primary Technical Purpose	Development Phase Used
Claude 3.7 Sonnet / Google Antigravity	Architectural brainstorming, drafting parser boilerplate, regex formulas, test scaffolding	Architecture, Ingestion, UI & Hardening
Groq Cloud (qwen/qwen3.8-27b)	Runtime LPU inference engine (300+ tokens/sec, sub-second latency)	Runtime Inference, Grounding & Evaluation
HuggingFace (BAAI/bge-small-en-v1.5)	Local dense vector embeddings inside persistent ChromaDB (zero cloud calls)	Vector Embedding & Persistent Retrieval Indexing

6.1 Human Insight Steering vs. AI Assistance (Leading the AI, Not Led by It)

- *Human Architectural Decisions:* Choosing Sub-track 1A based on visual document analysis; designing the multimodal chunk schema (chunks.json); formulating the Character Dossier (+0.35) and Multi-Entity (+0.40) re-ranking algorithms; authoring Directive 5 to eliminate narrative bridging; designing the 3-stage adaptive budgeting engine; and establishing a 100% free local architecture.
- *AI Acceleration Tasks:* Writing PyMuPDF image cropping loops, drafting regex patterns for citation extraction, generating Streamlit UI CSS layouts, and writing test assertions.
- *Human Intervention & Correction:* Human auditing caught the LLM fabricating speculative explanations for archive contradictions ('divergent narrative framing'). The team intervened, rejected the AI output, and hardcoded Directive 5.
- *Audit Trail:* Complete, unedited plain-text logs are preserved in ai_usage/claude.txt per Section 4.1.

7. Engineering Best Practices & Reproducibility (Section 5.2)

7.1 Repository Structure Matching Section 5.2 Guidelines

Our Git repository strictly matches the folder layout specified in the competition challenge document:

```
DeepThink/
|-- .git/                                # Full 2-week atomic commit history (28 Aug - 9 Sep 2026)
|-- .gitignore                          # Strict exclusion of virtualenvs, caches, and credentials
|-- README.md                          # Comprehensive setup, run, and architecture documentation
|-- docs/
|   |-- architecture.md                # Full system architecture specification
|   |-- decisions.md                  # Architecture decision log (13 ADRs)
|   |-- limitations.md                 # Known limitations & discarded approaches log
|   +-- diagrams/                     # Mermaid sequence & architecture diagrams
|-- src/
|   |-- config.py                     # Centralized configuration loader
|   |-- ingestion/                    # Multi-format parsing, OCR, table & figure extraction
|   |-- retrieval/                    # Router, budgeter, agentic search, reranker, verifier
|   |-- generation/                   # Grounded Groq LPU generator (Directive 5)
|   |-- evaluation/                   # Automated 20-question benchmark harness
|   +-- ui/                           # Interactive Streamlit web application
|-- tests/                             # Automated test suite (15 test modules, 76 tests)
|-- ai_usage/
|   |-- ai-usage-disclosure.md         # Mandatory AI Usage Disclosure (Section 4.1)
|   |-- claude.txt                    # Exported plain-text AI development logs
|   +-- context.md                    # System prompts, skills & constraints
|-- configuration-example/              # .env.example and config.example.json
|-- requirements.txt                    # Project dependency lockfile
+-- submission_report.pdf              # 5-page submission report
```

7.2 5-Step Reproducibility Guide for Judges

Judges can set up, verify, and run DeepThink completely from scratch with five simple shell commands:

```
# 1. Clone repository and navigate into folder
git clone https://github.com/IT25101463/DeepThink.git && cd DeepThink

# 2. Create virtual environment and install dependencies
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt

# 3. Configure free Groq API key (Obtain in 20s at console.groq.com/keys)
cp configuration-example/.env.example .env # Insert your GROQ_API_KEY

# 4. Run automated test suite (All 76 tests pass cleanly in ~55s)
python -m unittest discover -s tests

# 5. Run automated 20-question benchmark or launch web UI
python -m src.evaluation.evaluator # Evaluates 20 benchmark questions
streamlit run src/ui/app.py # Local: http://localhost:8501 | Cloud: https://deepthink-c9trxotutcgjenphr6ltkyl.streamlit.app
```

8. Conclusion & Enterprise Impact

DeepThink proves that enterprise-grade document intelligence does not require expensive proprietary cloud databases or fragile manual tuning. By combining **local dense vector search**, **modality-aware retrieval**, **autonomous adaptive budgeting**, **composite re-ranking**, and **lightning-fast LPU inference**, DeepThink delivers rich multimodal answers—embedding high-resolution diagrams and structured tables alongside verified citations—with **100% precision**, **zero hallucinations**, and **sub-second response times**.

Team DeepThink • SLIIT Codefest 2026 AI Competition • Initial Round Submission
Rasheed A. A. A. • Fatheen M. F. A. • M. M. M. Shakeer • S. Dharshan